

Vehicle Framework

Making a Vehicle

- VehicleBuildDef
 - Base Fields
 - Making the VehicleBuildDef
- VehicleDef
 - Base Fields
 - Making the VehicleDef
 - VehicleDrawProperties
 - VehicleProperties
 - components

Turrets

- VehicleTurretDef

Extra Comps

Performance

- Comps
- Pathfinding

Pattern Masking

Graphic Overlays

Custom Behavior for Vanilla

- CustomCostDefModExtension
- VehicleDamageMultiplierDefModExtension
- ProjectilePropertiesDefModExtension

Upgrades

Note

This document's purpose is to outline all features added or modified by the Vehicle Framework. Some things are not going to be explained in full since first and foremost this is meant to be a guide for those that want to create vehicles within the framework, but also because it would be a bit confusing (and add a lot of bloat) if I explained every mechanic or class in full detail when the majority of modders will be Xml only. Instead, I have documented the source code quite extensively in an attempt to add more clarity should any of you decide to look through it. The source code is on [GitHub](#) so you're more than welcome to go through it.

Feedback and contributions are welcome. If you have improvements or bug fixes, feel free to open a pull request (PR) — just make sure to contact me first as some changes may be too intrusive and will thus result in the PR being declined.

I also recommend joining the Starflight Creative [Discord](#) server, where I share updates, demos, and discuss potential changes.

Making a Vehicle

A Vehicle is composed of 2 defs. The first is a *VehicleBuildDef* which defines the building/frame of the vehicle. The second is a *VehicleDef* which defines the actual vehicle itself. There isn't a ton of information needed for the VehicleBuildDef since its lifetime is rather short lived (it only exists while a vehicle is in the process of being built) but I decided to keep them separately defined so modders can change the properties of this vehicle-in-progress, especially if ruins or wreckage come into play.

VehicleBuildDef

The *VehicleBuildDef* has a few key fields that must be defined to follow RimWorld's typical Building def. At the end of this section I will include an example screenshot.

Base Fields

For 99% of you, you'll first want to inherit from "VehicleBaseBuildable". This parent xml block will contain most of the important but obscure fields that you're unlikely to change, but for clarity sake let's go through it.

If you just want to skip through all the details and move on to the required fields outside of this parent xml block, then skip to [Making the VehicleBuildDef](#).

```
<Vehicles.VehicleBuildDef Name="VehicleBaseBuildable" Abstract="True">
  <thingClass>Vehicles.VehicleBuilding</thingClass>
  <selectable>true</selectable>
  <altitudeLayer>Building</altitudeLayer>
  <terrainAffordanceNeeded>Heavy</terrainAffordanceNeeded>
  <constructEffect>ConstructMetal</constructEffect>
  <leaveResourcesWhenKilled>true</leaveResourcesWhenKilled>
  <passability>Impassable</passability>
  <fillPercent>1</fillPercent>
  <neverMultiSelect>true</neverMultiSelect>
  <designationCategory>Vehicles</designationCategory>
  <clearBuildingArea>true</clearBuildingArea>
  <category>Building</category>
  <rotatable>true</rotatable>
  <blockWind>true</blockWind>
  <useHitPoints>true</useHitPoints>
</Vehicles.VehicleBuildDef>
```

The **thingClass** should remain as Vehicles.VehicleBuilding or some subclass of it. This is required, the reference to what VehicleDef or VehiclePawn it contains is necessary.

selectable is a flag for if the player is able to select the frame / building in-game. It's best to leave this as true, otherwise the player will find it difficult to cancel the building and impossible to view its progress.

altitudeLayer is for rendering, unless you're familiar with altitude layers it's best to leave this be.

terrainAffordanceNeeded defines what ground it is able to be built on. TerrainDefs have affordances, so this field essentially says "this vehicle can be built on any ground with at least X affordance." In most cases this will be Heavy, but you can also use others if you find it more fitting. Some sensible terrain affordances:

- Light
- Medium
- Heavy
- ShallowWater

constructEffect is an effect specific to building things. Default is ConstructMetal but you'll likely want to change this for vehicles of different build types (eg. wood).

Here are the suitable vanilla effects for construction (for full list of EffecterDefs, look them up.. There's like 90+ but not all are suitable for construction)

- ConstructDirt
- ConstructWood
- ConstructMetal
- RoofWork
- Repair

leaveResourcesWhenKilled this will only matter if you intend to have your vehicles show up as wreckage. When that wreckage is destroyed, this will determine if it drops a fraction of the costList or not.

passability: can pawns pass over / through it? (Impassable, PassThroughOnly, Standable)

fillPercent: how "full" is the frame. This mainly determines cover, but also things like temperature leakage are influenced by fillPercent.

neverMultiSelect: self explanatory, dictates if the player can select multiple frames of this building or not

designationCategory: Where this should show up in the architect menu.

If you inherit from “VehicleBaseBuildable” and don’t want it to show up in the architect menu then you’ll want to add this as a self closing node. It’ll set the designationCategory to null and make sure the XmlInheritance step on game startup doesn’t pull the default Vehicles architect tab.

```
<designationCategory/>
```

clearBuildingArea: move things that are on top of the build site out of the way

category: It’s best if you don’t change the thing category. This is a building, it shouldn’t be anything else.

rotatable: Should the vehicle be able to be rotated and built in a different direction? For non-rotating vehicles such as a hot air balloon or a rocket, this should be set to false.

blockWind: Only for wind turbines and if this frame / building will prevent the turbine from getting proper wind flow. This should (in most cases) always be true. Vehicles are big, and if rock chunks can stop it, then so should vehicles :)

useHitPoints: It’s a building, and buildings use hitpoints. Don’t worry about vehicle health for this part. This is just the building portion of it.

Making the VehicleBuildDef

Follow the same procedure you would when making a good ol’ vanilla building (like a wall). Since the building will never actually be spawned apart from maybe wreckage / ruins, take this into consideration when thinking about adding comps or other custom behaviors. This building is a *precursor to spawning the vehicle* so right now it won’t ever appear in-game.

There is only 1 field that needs to be defined for the VehicleBuildDef that is not vanilla.

thingToSpawn: This should point to the VehicleDef this building will spawn when it’s finished constructing

soundBuilt: If you want an extra one shot to play besides just Building_Complete, then you can specify what sound.

Here is what the finished def looks like.

```
<Vehicles.VehicleBuildDef ParentName="VehicleBaseBuildable">
  <defName>Tank_Blueprint</defName>
  <label>Tank</label>
  <description>A heavily armored combat vehicle.</description>
```

```

    <graphicData>
        <texPath>Vehicles/Land/Tank/Tank</texPath>
        <graphicClass>Graphic_Multi</graphicClass>
        <drawSize>(4,7)</drawSize>
    </graphicData>

    <statBases>
        <MaxHitPoints>5000</MaxHitPoints>
        <WorkToBuild>125000</WorkToBuild>
        <Flammability>0.05</Flammability>
    </statBases>

    <size>(3,5)</size>

    <researchPrerequisites/>

    <costList>
        <ComponentIndustrial>35</ComponentIndustrial>
        <Steel>1500</Steel>
    </costList>

    <soundImpactDefault>BulletImpact_Metal</soundImpactDefault>

    <thingToSpawn>Tank</thingToSpawn>
</Vehicles.VehicleBuildDef>

```

VehicleDef

This is the most complicated part of creating a vehicle. The VehicleDef is what defines the actual vehicle itself. It is most similar to creating a pawn, except that you don't have to define a PawnKindDef. The VehiclePawn (the vehicle object in-game) is a pawn by technicality, but its mechanics and behavior is different in almost every way. It's important to keep this in mind as you will be able to use and define the same fields that normal pawns do, but not all mechanics will work the same (if at all). eg. VehiclePawns do not have body parts, needs, tools, or even hediffs. Keeping this in mind, if you're not familiar with C# and/or don't have the patience to test what mechanics may or may not carry over from normal pawn behavior, stick to the guide for what vehicle-specific fields to include.

Base Fields

To start out, you'll want to either inherit from "BaseVehiclePawn" or be sure to include all of the fields defined in that abstract def. All of the fields defined there are necessary (apart from

stats). Unless you're familiar with ThingDefs and what fields are needed for Pawn races, it's best to not try and reinvent the wheel here.

Note: Right now it is best to not change passability from Standable to anything else. Since the Steam Deck update, there have been crashes as a result of pawns attempting to path somewhere deemed impassable. This is not easily solvable as passability is checked through region reachability, which in turn can cause vehicles to fail reachability checks themselves. This can also cause tremendous lag when vehicles block common pathing areas (or jobs blocked by a vehicle such as repairing a wall). In other words, no touchy until I say otherwise. Give the vehicle a high pathCost and be on your way.

```
<comps>
  <li>
    <compClass>CompAttachBase</compClass>
  </li>
</comps>

<drawGUIOverlay>true</drawGUIOverlay>
```

Making the VehicleDef

graphicData (Required)

The graphic class for vehicles is required to be `Graphic_Vehicle` (*Vehicles.Graphic_Vehicle* fully typed out). This is non-negotiable, as the vehicle graphic class handles not only 8 directional rotations (allowing for vehicles to render diagonally), but also RGB, pattern, or skin shader types. Even if you don't intend to use any complex shaderType, or even diagonal rendering, it must be set to *Vehicles.Graphic_Vehicle*.

enabled

If you want to restrict who your vehicle can be used by then set this field to that restriction. By default, vehicles are allowed to be used by everyone. When raiders / traders are implemented, they will pull from whatever vehicles they are allowed to use. If a vehicle is marked as not usable to the player, it will be hidden from the architect menu, but not removed from the game. This technically means the player can still obtain the vehicle should they capture it from raiders (assuming enabled = Raiders).

- Everyone
- Raiders
- Player
- None

nameable

Vehicle can be assigned a name. The name will overlay on top of the vehicle just like it does for colonists.

buildDef (Required)

References the VehicleBuildDef responsible for building this vehicle.

combatPower

Normally this would be defined in the PawnKindDef, but I've taken the liberty to just create an implied def for any vehicle that is not explicitly assigned a PawnKindDef. Why? Because there is quite literally only 1 field that matters in the PawnKindDef when it comes to vehicles, and that is combatPower. This value is in relation to point usage when generating NPCs.

fillPercent

Most vehicles will want to be 1 fillPercent. This ensures that vehicles provide full cover much like walls do. If you only want your vehicle to provide partial cover, say like a bicycle or moped, then you'll wanna give it < 1 fillPercent. The value you give is the coverage effectiveness, eg. sandbags have a coverage of 55% (or 0.55 in xml).

size

Size of the vehicle in (x,y) format.

uilconScale

Scale of vehicle icon displayed in-game. If no ThingDef icon is provided (overrides original behavior where vehicle is rendered instead), the image will still scale based on this value.

vehicleMovementPermissions

Movement permissions determine what conditions a vehicle must satisfy before being able to move.

- DriverNeeded (all roles with Movement handling type must be filled to operational count)
- NotAllowed (vehicle is not able to be moved, ever. Used for aerial vehicles that shouldn't move)
- NoDriverNeeded (vehicle is able to move without any pawns or roles being filled)

vehicleCategory

Category of the vehicle in regards to its primary use. This is unused at the moment but will serve as a way to filter vehicles out for NPCs. (eg. Raiders should focus on primarily bringing combat or hybrid vehicles, rather than ones specifically for traders)

You don't *have* to set this right now, but keep in mind in the future that this will be used for Raiders / Traders and enemy vehicle generation. This field is subject to change depending on the future update for NPCs.

- Misc
- Transport
- Trader
- Combat
- Hybrid

techLevel

Same techLevel of other ThingDefs in RimWorld. The important part of defining this for VehicleDefs is that factions will not spawn with vehicles above their techLevel. (eg. you won't see tribals spawning with tanks)

vehicleType

While vehicles are able to be given all kinds of custom pathing (including hybrid vehicles that can path on both water and land), vehicles still fall into a specific category. This category lumps vehicle types together and prevents non-overlapping behavior (such as the player attempting to form a caravan with both a boat and land vehicle). In the future, I may implement this as a flagged enum (let's you specify more than 1 vehicleType) but this is TBD.

- Land
- Air
- Sea
- Universal (will be removed if enum is implemented with flags)

navigationCategory

How the vehicle travels on the world map during nighttime. Right now there are 3 navigation types, with only the 3rd one making a noticeable difference. Later on I intend on making every type have its own behavior. This may or may not be turned into a def at a later point depending on other modders' needs.

- Manual
- Opportunistic
- Automatic (will travel in caravans 24/7 without rest)

statBases

Normal vanilla stats that apply to vehicles. Note: don't use vanilla MoveSpeed, but rather the vehicle stat MoveSpeed. Vehicle stats are not only stripped down versions of vanilla stats, but also hooked up with vehicle components. Using vehicle-specific mechanics rather than trying to squeeze it in through vanilla mechanics (StatDefs + BodyParts) was much more efficient and clean.

Relevant Vanilla StatDefs

- Mass
- ArmorRating_Blunt
- ArmorRating_Sharp
- ArmorRating_Heat
- MarketValue
- Flammability

vehicleStats

VehicleStatDefs are stripped down versions of vanilla StatDefs. They're also able to be hooked up to VehicleComponents (the parts that make up the vehicle). As a VehicleComponent takes damage, that component's efficiency % will also go down. This means stats the component is responsible for will also take a hit. eg. a tire being shot out will directly affect MoveSpeed.

Relevant VehicleStatDefs:

- MoveSpeed
- CargoCapacity
- RepairRate
- BodyIntegrity (integrity of the vehicle. If BodyIntegrity efficiency = 0%, the vehicle will disintegrate)
- RepairRate
- FlightSpeed
- FlightControl (TBD)

VehicleDrawProperties

Defines rendering properties related to UI and overlays. This is mostly fluff but can be useful for customizing your vehicle.

displayRotation

Rot8 rotation the vehicle is rendered in for UI. Some examples would be the mod settings, gizmos, and icons. Rot8 is similar to Rot4 (RimWorld's struct for defining thing rotations) except that it also defines diagonals. Default rotation is East, similar to how animals are rendered in UI.

- North
- South
- East
- West
- NorthEast
- SouthEast
- SouthWest
- NorthWest

Note: This will only work with shader types *CutoutComplexRGB* and *CutoutComplexPattern* which both require masks. This is because a new material request is created that pulls the material for that rotation from the graphic class. In the future, I may tweak this to also work with vanilla shader types, although if you're using anything but *Cutout* (you have no masks) then you should be using *CutoutComplexRGB* (same as CutoutComplex but supports a blue channel for masking) or *CutoutComplexPattern*.

displayOffset

Similar to graphicData's draw offsets, this will offset the texture in UI. Due to scaling and sometimes vehicles not being centered at the exact center of the vehicle texture, sometimes you'll need to make slight adjustments to the rendered position.

displaySizeMultiplier

Increase or decrease the size of the texture when rendered in UI. Multiplier is a single decimal value, which will be applied to both dimensions. This means you can't stretch or squeeze down the dimensions of the texture, only scale them.

loadCargoTexPath (Optional)

Texture path for the icon to show for the "Load Cargo" gizmo.

cancelCargoTexPath (Optional)

Texture path for the icon to show for the "Cancel Loading" gizmo.

Note: Just leave the above two blank if you wish to use the default textures provided by the framework.

graphicOverlays

List of graphic overlays to render on top of your vehicle. This field itself is a list of GraphicDataOverlays.

GraphicDataOverlay

The graphic data overlay class is currently a small placeholder for future additions. Right now, it contains a GraphicDataLayered field, and a rotation field. There may be more fields in the future for more customization, but for now these are the only 2.

rotation

This is the rotation to render the texture in (0 to 360). One example would be helicopter rotors, where you may use the same texture for the main rotor blade and the tail rotor but want to render them at 2 different angles.

graphicData (type:GraphicDataLayered)

GraphicDataLayered is the same as vanilla's GraphicData, except that you also specify a layer for the texture to be rendered.

Unity is a 3D engine, and even though RimWorld is a 2D game it is still rendered in 3D space. You could specify a drawOffset for the y-axis (to and from the camera in RimWorld's case) but that can be tedious and confusing if you don't know the altitude

increment. This integer value takes all layers from the AltitudeLayer enum and calculates a valid offset such that your textures will be layered properly without bleeding into the next altitude layer, which would screw up the rendering order of your overlay. (eg. your overlays could end up rendering over motes, skyfallers, and even fog if you screwed up the y offset).

Note: Larger layer values will be rendered further towards the camera (on top) and smaller layer values will be rendered further away from the camera (on bottom). So if you had 3 overlays (A, B, and V) with layer values 1, 2, and 3 respectively. The draw order would be C on top, B in the middle, and A on bottom. If 2 textures share the same layer, they will be rendered at the same y coordinate and the draw order will be determined by Unity.

Graphic overlays don't have to be specifically something that is to be layered on top of the vehicle (like helicopter rotors). It can be used for any extra textures you wish to render alongside your vehicle, and if you do not specify a layer it will default to 0.

VehicleProperties

Additional vehicle properties for specific mechanics ingame. The field in XML is named `properties`.

visibility

Vanilla RimWorld determines visibility of a caravan (and its pawns) by the pawns' body size. Because that doesn't make too much sense for vehicles, and also because body size affects more than just caravan visibility, this field is separated so you can explicitly define how much this vehicle should affect caravan visibility. Apart from this replacing the calculation based on body size of an individual pawn, the visibility formula for caravans will remain the same as vanilla.

vehicleJobLimitations

Vehicles are something that in practice, will likely take a long time to build and repair. Rather than letting people do this for immersion at the cost of ruining gameplay experience (no one wants to spend an entire quadrum repairing a car), you may specify how many pawns are allowed to work a vehicle specific job at a time. This is currently restricted to certain jobs only, since the job drivers have to take the max workers into account.

- RepairVehicle
- UpgradeVehicle (Inactive at this time)
- LoadUpgradeMaterials (Inactive at this time)

If you leave this field blank, it will default to 2 workers for each job.

diagonalRotation

Vehicle should rotate diagonally (or use diagonal textures) when traveling in non-cardinal directions. Defaults to true and should be left alone in most cases. For small vehicles (1x1) it may be better to use 4 directional sprites for traveling non-cardinally.

- true / false

manhunterTargetsVehicle

Manhunter animals should target this vehicle to attack it. Vehicles are a subtype of pawns after all, so they are technically pulled as potential candidates to attack when a manhunter pack enters the map. Luckily, I disabled this behavior (cause it makes no sense) but if you actually want it, then set this to true. Otherwise, this will be false by default.

- true / false

iconTexPath

generateThingIcon

Health Labels

Health labels are what show up in a pawn's inspector pane in the bottom left hand corner (must be actively selected). The *BaseVehiclePawn* xml parent fills these out already so these are technically optional, but if you decide to not inherit from *BaseVehiclePawn* you'll want to fill these out.

Pathing

Pathing for vehicles includes both the world map and local maps. If you wish to make specific terrain, roads, biomes, etc. impassable, there are 2 different limits for what is considered an "impassable" path cost. Assigning a path cost equal to or higher than this max limit will make the game register it as impassable. The limits are:

World Map: 1000

Local Map: 10000

defaultTerrainImpassable

Sea-type vehicles only

fishing

Allows vehicles to fish when on valid fishing tiles and a compatible fishing mod is in the active mod list.

- true / false
- Currently disabled until fishing compatibility is reimplemented

wakeMultiplier

Size multiplier of a vehicle's wake (the waves that appear behind boats)

wakeSpeed

Speed of a vehicle's wake

components

This is a list of VehicleComponentProperties. These are, essentially, the vehicle's body parts - although, instead of a tree, vehicles use a different system based on part depths and cells.

Properties of a VehicleComponent are:

key

Essentially, the part's defName. These need to be unique per vehicle.

label**health****depth**

External parts with a hitbox in the hit location are always hit first, then internal parts in the hit location, then all internals, then anything else.

- External
- Internal
- Undefined

hitWeight (Optional, defaults to 1)

Essentially, the chance for a part to be hit - parts with hitWeight set to 2 are twice as likely to be hit, for instance.

armor (Optional)

```
<armor>
  <ArmorRating_Blunt>0</ArmorRating_Blunt>
  <ArmorRating_Sharp>0</ArmorRating_Sharp>
</armor>
```

This does not add together with the vehicle's own armor, but overrides it, which means that setting it lower than the vehicle's armor ratings makes the part less armored.

hitbox

```
<hitbox>
  <fallthrough>>false</fallthrough>
  <cells>
    <li>(0, 2)</li>
  </cells>
```

```
</hitbox>
```

Note that if a part doesn't set its hitbox, it's automatically set to be in the $(0, 0)$ cell!

cells

A list of all cells in the part. Overrides any other cell settings if set.

side

Automatically sets the part's cell to a side of the vehicle.

- **Empty** $(0, 0)$
- **Front, Right, Back, Left**
- **Body** (the entire vehicle)

fallthrough (Optional , defaults to true)

An inverted equivalent of solid in a body part: setting it to false means damage doesn't pass to other parts if the part is destroyed by a hit.

- true / false

reactors (Optional)

A list of parts that react to the part being hit. Vehicle framework comes with two reactors, Reactor_FuelLeak and Reactor_Explosive.

indicator

```
<indicator>VF_Explosive</indicator>
```

A def with the indicator icon in it.

highlightColor

```
<highlightColor>(1.0, 0.5, 0)</highlightColor>
```

Reactor_FuelLeak

A part with this reactor will leak fuel if its health falls under a certain threshold.

maxHealth

```
<maxHealth>0.8</maxHealth>
```

Health percentage when the part starts (or stops) leaking.

rate

```
<rate>1~10</rate>
```

The rate of fuel leaking, in units per second. The first value is when the part's health percentage is at maxHealth and the second is when the part is at 0 health.

Reactor_Explosive

A part with this reactor has a chance to explode per hit.

maxHealth

```
<maxHealth>0.8</maxHealth>
```

chance

```
<chance>0.25</chance>
```

damage**armorPenetration****radius****damageDef****categories (Optional)**

A list of stats the part lowers as it gets damaged. Equivalent of tags in BodyPartDef, but simplified.

efficiencyWeight (Optional, defaults to 1)

How much the part's damage affects the overall stat; only important for stats that use the average of all part efficiencies.

priorityStatEfficiency (Optional)

If set to true, then damage to this part will affect the vehicle as if the vehicle only has parts with priorityStatEfficiency set to true.

efficiency (Optional)

```

<efficiency>
  <points>
    <li>(0, 0)</li>
    <li>(0.25, 0)</li>
    <li>(0.4, 0.4)</li>
    <li>(0.7, 0.7)</li>
    <li>(0.85, 1)</li>
    <li>(1, 1)</li>
  </points>
</efficiency>

```

Part's health affects its efficiency on a non-linear curve; you can change the curve if you really need to.

tags (Optional)

They don't currently do anything.

compClass (Optional)

If you need a part to do something really custom, you can subclass VehicleComponent and put it here.

Turrets

Note - these turrets are a Vehicle Framework implementation and do not use the RimWorld verb system.

VehicleTurretDef

<turretType>

Description	Defines whether the turret is fixed or can rotate.
Type	Static Rotatable
Default	Rotatable

Example:

```
<turretType>Rotatable</turretType>
```

<ammunition>

Description	Filters the ammo types allowed.
Type	ThingFilter
Default	null

Example:

```
<ammunition>
  <thingDefs>
    <li>Steel</li>
  </thingDefs>
</ammunition>
```

<magazineCapacity>

Description	Number of shots before needing to reload.
Type	integer

Default	1
Note	Set to 0 to disable the need for reloading.

Example:

```
<magazineCapacity>60</magazineCapacity>
```

<chargePerAmmoCount>

Description	Amount of resource (ammo or material) per magazine count.
Type	decimal
Default	1.0

Example:

```
<!--  
Requires 7.5 steel for 1 shot, rounded up.  
eg. Reloading 10 count magazine requires 75 steel,  
11 requires 82.5 steel, rounded up > 83  
-->  
<magazineCapacity>7.5</magazineCapacity>
```

<genericAmmo>

Description	Turret consumes non-projectile resources (e.g., steel, chemfuel) instead of standard ThingDef with projectile properties.
Type	true false
Default	false

Example:

```
<!-- Requires 5 steel for 1 shot -->  
<magazineCapacity>5</magazineCapacity>
```

<cooldown>

Description	Optional cooldown properties, limiting the number of shots before needing a break from firing to cooldown.
-------------	--

Type	TurretCooldownProperties
Default	null

Example:

```
<cooldown>
  <heatPerShot>2.5</heatPerShot>
  <dissipationRate>0.5</dissipationRate>
  <dissipationCapMultiplier>1</dissipationCapMultiplier>
</cooldown>
```

<fireModes>

Description	Configure selectable fire modes
Type	List<FireMode>
Default	null
Note	At least 1 must be defined

Example:

```
<fireModes>
  <li>
    <shotsPerBurst>1</shotsPerBurst>
    <spreadRadius>4</spreadRadius>
    <label>Single</label>
    <texPath>UI/Gizmos/FireRate_Single</texPath>
  </li>
</fireModes>
```

<autoSnapTargeting>

Description	Snap instantly to target instead of rotating at a fixed rate.
Type	true false
Default	false

Example:

```
<autoSnapTargeting>true</autoSnapTargeting>
```

<rotationSpeed>

Description	Speed (degrees per second) at which the turret rotates to track targets.
Type	decimal
Default	1
Note	If autoSnapTargeting is true, rotationSpeed will be ignored.

Example:

```
<rotationSpeed>3.1</rotationSpeed>
```

<maxRange>

Description	Maximum range for targeting enemies.
Type	decimal
Default	0
Note	Set to 0 to disable max range constraint.

Example:

```
<maxRange>25.9</maxRange>
```

<minRange>

Description	Minimum range for targeting enemies.
Type	decimal
Default	0
Note	Set to 0 to disable min range constraint.

Example:

```
<minRange>5.9</minRange>
```

<reloadTimer>

<reloadTimerMultiplierPerCrewCount>

<warmUpTimer>

<autoRefuelProportion>

<empDisables>

<restrictionType>

<recoil>

<vehicleRecoil>

<motes>

Description	A list of visual effects (motes) spawned after firing. Use for muzzle flashes, smoke, sparks, etc.
Type	List<AnimationProperties>
Default	null

Example:

```
<motes>
  <li>
    <cycles>1</cycles>
    <moteDef>Mote_CannonPlume</moteDef>
    <offset>(0, 0, 2)</offset> <!-- z axis, or forward from turret -->
    <animationType>Reset</animationType>
  </li>
</motes>
```

<graphicData>

<graphics>

<matchParentColor>

<gizmoDescription>

<gizmoIconScale>

<gizmoIconTexPath>

<shotSound>

<reloadSound>

<targetScanFlags>

<projectile>

<attachProjectileFlag>

<projectileOffset>

<projectileSpeed>

<projectileShifting>

Extra Comps

Performance

This project has been optimized fairly well so that people can run it no problem with high counts of vehicles, large colonies, and large maps. It has virtually no impact on games without any vehicles spawned in, and very minimal impact for vehicles that are just idling.

If you're making a vehicle mod and plan on extending off the code base (and not just XML) it would be wise to pay attention to the following:

Comps

Vanilla comps tick constantly by default, hogging up precious resources oftentimes for no reason. VehicleComps can notify the parent it will be ticking on request. This means it will only

tick while necessary, and request to stop ticking as soon as it's done. A good example of this would be `CompFueledTravel`, which only ticks when it is actively using fuel which is then validated by events related to moving, drafting, etc. The status for which it wants to consume fuel will only change during those events, so it makes sense to only validate and begin ticking after those events trigger. The result is 0 impact from this comp while the vehicle is doing nothing at all.

Pathfinding

I recommend not trying to mess around with regions and pathfinding for vehicles, as it is quite entangled and complex. On top of this, it also exclusively runs on a dedicated thread which is created for each map. This means any methods or patches you write that will be digging into the pathfinding of vehicles **must be thread safe**. I have exposed a lot of configuration settings to xml for vehicle pathfinding, so there should be no need to touch this section of code. If you do feel it is necessary, please notify me so I can at least be aware of such changes, as breaking vehicle pathfinding will end up being reported to me at some point anyways.

Pattern Masking

Graphic Overlays

Custom Behavior for Vanilla

One of the great things about this framework is the flexibility in designing vehicles exactly how you want them to be, without actually affecting vanilla content. There are a few `DefModExtensions` for this exact reason. By adding or patching these `DefModExtensions` onto other defs, you can give them custom behavior with vehicles without affecting the values they might have in normal circumstances.

CustomCostDefModExtension

If you need something to have a specific `pathCost` for just vehicles (without also applying that path cost to vanilla pathfinding), this is the go to method. In this `DefModExtension` there are 2 fields:

1. **vehicles** - a list of `VehicleDefs` this should apply to (if not specified / left blank, it will apply to all `VehicleDefs`)

2. **cost** - the path cost you want it to use. This DefModExtension is able to be placed on TerrainDefs and ThingDefs for local maps, as well as BiomeDefs, RiverDefs, and RoadDefs for the world map.

VehicleDamageMultiplierDefModExtension

This one is a bit more niche, as vanilla combat (and damage values) are functional without any external changes. There are 3 sliders in the mod settings for tweaking melee, ranged, and explosive damage as a generic way for balancing vehicle combat to your liking, but that is for players not modders. This DefModExtension gives a multiplier to the weapon's damage value before it's affected by the mod settings multipliers, and before damage falloff from armor and deflection is applied. If you truly want to give your weapon more of a punch against vehicles without having to give it a high damage or armor penetration value, then you can use this. It's also worth mentioning that this can be placed on any ThingDef, as instigators of damage will also check for this extension.

ProjectilePropertiesDefModExtension

Out of all the DefModExtensions, this is probably the most useful in regards to vanilla content. Rather than diverting behavior away from vanilla and only applying to vehicles, this DefModExtension modifies projectiles launched from vanilla turrets. A great example is the flak 88 I created in the closed beta. Vanilla projectiles are given something called hit flags. These hit flags determine when a projectile should be intercepted, if at all, and hit the thing it was intercepted by or continue onward to the intended target. This would be great and all, however projectile hit flags are specific to the projectiles and not the launcher. This means if I were to try and use vanilla projectiles (such as those assigned in mortar shells) they would launch with a slow speed and an overhead hit flag. The flak 88 would be constrained to normal mortar behavior simply because I wanted to use the same ammunition.

This DefModExtension allows you to change the properties of the projectile immediately before it's launched. While this is also quite niche since it's technically unrelated to vehicles, I find it useful nonetheless. The flak 88, paired with the recoiling animation, is one of my favorite test objects I've made for any of my mods. The VehicleTurret has this same behavior except the modification to vanilla projectiles is built in to match the projectile speed and hit flags specified in the VehicleTurretDef.



Upgrades

Vehicle Framework - Upgrades